

Service Overview

ภาพรวมของบริการ (Service Overview)

Manage Dispatch Service

Github: <https://github.com/Slasherw/ManageDispatchService>

Video Demo:  CS366 ManageDispatch Demo

1. Service Owner

นายณพกรณ์ สงเคราะห์ รหัสนักศึกษา 6609540023

2. Service Purpose

Manage Dispatch Service เป็นบริการที่รับผิดชอบการบริหารจัดการคำสั่งการกู้ภัย โดยทำหน้าที่เชื่อมโยงทีมกู้ภัย (Rescue team) ที่เฉพาะเจาะจงเข้ากับคำร้องขอความช่วยเหลือ (Request) รวมถึงติดตามสถานะการร้องขอทั้งหมดที่เกิดขึ้นในเหตุการณ์ภัยพิบัติ เพื่อให้ส่วนกลางสามารถเฝ้าระวังสถานะการทำงานได้อย่างมีประสิทธิภาพ

3. Pain Point ที่แก้ไข

ในช่วงเหตุการณ์ภัยพิบัติที่มีความโกลาหล ศูนย์สั่งการ (Dispatchers) มักจะสูญเสียการติดตามสถานะของหน่วยกู้ภัยที่ถูกส่งออกไปปฏิบัติหน้าที่ ซึ่งปัญหาดังกล่าวนำไปสู่การส่งทีมกู้ภัยไปทำงานซ้ำซ้อนในพื้นที่เดิม หรือทำให้มีบางพื้นที่ที่ต้องการความช่วยเหลือถูกละเลยไป

4. Target Users

- เจ้าหน้าที่ศูนย์สั่งการ / ผู้สั่งการ (Dispatcher)
- บริการนี้ทำหน้าที่เป็น Backend Service หลักที่เชื่อมต่อระหว่าง Request Service, Team Service และ Mission Progress Service

5. Service Boundary

In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ):

- จัดเก็บและจัดการข้อมูลคำสั่งการ (Dispatch Records)
- จัดการส่งมอบ Request ให้กับ Team โดยใช้ข้อมูลคำแนะนำทีมหรือข้อมูลอื่นของทีมเพื่อการตัดสินใจ
- บังคับใช้ตรรกะการตรวจสอบความพร้อมของทีมกู้ภัยก่อนลงพื้นที่
- อัปเดตสถานะการทำงานของทีม (BUSY, AVAILABLE) และสถานะของ Request (TRIAGE, ASSIGN)

Out-of-scope / Not Responsible For (ไม่รับผิดชอบ):

- การจัดการข้อมูลพื้นฐานของทีมกู้ภัย (Team Master Data)
- การจัดการรายละเอียดเชิงลึกของคำร้องขอความช่วยเหลือจากประชาชน (Request/Incident Master Data)

6. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การตรวจสอบและตัดสินใจอนุมัติคำสั่งการ โดยอิงจากสถานะ "AVAILABLE" ของทีมกู้ภัยจาก Team Service

- การปฏิเสธการสร้างคำสั่งการ (Dispatch) หากอ้างอิงถึงข้อมูลที่ไม่มีอยู่จริง (เช่น ไม่มี Incident ID) เพื่อป้องกันการเกิดข้อมูลกำพวด (Orphan data) ในระบบ
- การจัดการสถานะ Flow การทำงาน เช่น การเปลี่ยนสถานะจาก WAITING เป็น DISPATCHED

• 7. Owned Data

- Dispatch Records:

- ข้อมูลหลักที่บริการนี้เป็นเจ้าของและจัดเก็บใน DynamoDB ได้แก่:

- dispatchId: ไอเดียหลักของการสั่งการ (Primary Key)
- requestId: ไอเดียอ้างอิงจากระบบ Request Service
- status: สถานะปัจจุบัน (เช่น WAITING, DISPATCHED, RESOLVED, CANCELLED)
- teamId: ไอเดียของทีมที่ได้รับมอบหมายอ้างอิงจาก Team service
- teamDetails: รายละเอียดทีม (ที่ดึงมาจาก Team Service ณ เวลาที่สั่งการ)
- updatedAt / dispatchedAt: เวลาที่มีการอัปเดตสถานะล่าสุดหรือเวลาที่ส่งทีมออกไป
- statusNote: บันทึกเพิ่มเติมจากการเปลี่ยนสถานะ (เทียบเท่ากับ completion_note ในเอกสารเก่า)

- Mission Acceptance State:

- สถานะของการสั่งการที่บริการนี้ควบคุม ได้แก่การเปลี่ยนผ่านจาก WAITING ไปเป็น DISPATCHED (เมื่อเจ้าหน้าที่ยืนยัน) และจาก DISPATCHED ไปเป็น RESOLVED (เมื่อได้รับแจ้งว่าภารกิจสำเร็จ)
- หมายเหตุ: ปัจจุบันระบบเราครอบคลุมถึงสถานะ CANCELLED (การยกเลิก) ด้วย

8. Linked Data (Reference Only)

- requestId: อ้างอิงจาก Request Service เพื่อระบุว่าการสั่งการนี้ตอบสนองต่อคำขอช่วยเหลือฉบับใด
- teamId: อ้างอิงจาก Team Service เพื่อระบุว่าทีมผู้ปฏิบัติงานใดเป็นผู้รับผิดชอบภารกิจ (มีการดึงข้อมูลรายละเอียดทีมมาเก็บแบบ Snapshot ในระบบในขั้นตอน Dispatch)
- dispatchStatus / missionProgress: ข้อมูลสถานะภารกิจในเชิงรายละเอียด (เช่น "ถึงที่หมายแล้ว", "กำลังช่วยเหลือผู้ป่วย") จะถูกควบคุมโดย Mission Progress Service ซึ่งจะส่งการอัปเดตกลับมายัง Manage Dispatch Service ผ่านทาง API PATCH /v1/dispatches/{id}/status เมื่อภารกิจสิ้นสุด (RESOLVED)

9. Non-Functional Requirements

• Idempotency & Data Integrity:

- ระบบใช้ dispatchId (UUID) เป็นกุญแจหลักในการอัปเดตข้อมูล และมีการใช้ ConditionExpression="attribute_exists(dispatchId)" ใน DynamoDB เพื่อป้องกันการอัปเดตข้อมูลที่ไม่มีอยู่จริง

- หากมีการเรียกใช้ API กับ ID ที่ไม่มีในระบบ ระบบจะตอบกลับด้วย HTTP 404 Not Found ตามมาตรฐานที่กำหนดไว้

• Resilience & Failure Handling:

- External Calls: ระบบมีการจัดการ Exception สำหรับการเรียกไปยัง External Service (Team Service, Request Service) โดยกำหนด Timeout (5 วินาที) เพื่อป้องกันระบบค้าง (Blocking)

- Fallback Logic: หากไม่สามารถดึงข้อมูลทีมจาก Team Service ได้ ระบบจะบันทึก Log ข้อผิดพลาดและดำเนินการต่อโดยใช้ข้อมูลพื้นฐานที่มี (Graceful Degradation) เพื่อไม่ให้เกิดการหยุดชะงัก

- Traceability:
 - ระบบรองรับการส่งและรับ X-Trace-Id ในทุก Request เพื่อให้สามารถติดตามการทำงานข้าม Microservices ได้ง่ายเมื่อเกิดปัญหา
- Security & Access Control:
 - มีการกำหนด CORS Policy ที่เข้มงวด (อนุญาตเฉพาะ Methods และ Headers ที่จำเป็น)
 - รองรับการใช้งาน LabRole ใน AWS เพื่อจำกัดสิทธิ์การเข้าถึง Resources (IAM Policy) ให้เฉพาะเจาะจงตามหลัก Least Privilege

Sync Contract

Synchronous Function Contract

Base URL: <https://qi7ip5zv5a.execute-api.us-east-1.amazonaws.com/Prod>

API Contract #1: [Process] Internal Dispatch Creation

หมายเหตุ: ในระบบปัจจุบัน ขั้นตอนการทำงานแบบ Asynchronous ผ่าน SQS Trigger ไม่ใช่ HTTP POST

Trigger: SQS Message (from Team Service SNS)

Worker: RecommendationWorkerFunction

สถานะเริ่มต้น: WAITING (ไม่ใช่ PENDING)

การทำงาน:

1. รับข้อมูลการแนะนำทีม (AI Recommendation) จาก Team Service
2. ดึงข้อมูลรายละเอียดทีมแบบละเอียด (Enrichment) มาเก็บลง Database
3. เรียก Request Service (Action: triage) เพื่อเปลี่ยนสถานะเป็น TRIAGED

- Idempotency: Yes (ใช้ requestId เป็น Primary Key ในการสร้าง Record)

API Contract #2: Get Dispatches

Name: Get dispatches for Dashboard or Team App

Method: GET

Path: /v1/dispatches

Request Query Params:

- teamId (string, optional): กรองตามไอดีทีม
- status (string, optional): กรองตามสถานะ (เช่น WAITING, DISPATCHED)

Response (Success 200):

```
{
  "items": [
    {
      "dispatchId": "uuid-string",
      "requestId": "uuid-string",
      "status": "WAITING",
      "priorityLevel": "HIGH",
      "dispatchedAt": "2024-05-18T10:00:00Z"
    }
  ],
  "traceId": "trace-id-string",
  "serverTime": "2024-05-18T10:05:00Z"
}
```

Dependency: ไม่เรียก Service อื่น (อ่านจาก DynamoDB โดยตรง)

API Contract #3: Update Dispatch Status

Name: Update dispatch and trigger external services

Method: PATCH

Path: /v1/dispatches/{id}/status

คำอธิบาย: ใช้เปลี่ยนสถานะการส่งทีม และสั่งการไปยังระบบภายนอก (Team/Request Service) โดยอัตโนมัติ

Body:

```
{
  "status": "DISPATCHED",
  "teamId": "TEAM-005",
  "note": "ส่งทีมกู้ภัยด่วน"
}
```

Allowed Status: DISPATCHED, RESOLVED, CANCELLED

Logic & Dependency:

1. If status = DISPATCHED:
 - เรียก Team Service (PATCH /status) เพื่อตั้งทีมเป็น BUSY
 - เรียก Request Service (POST /assign)
 - Publish SNS Event: DispatchOrderCreated ไปยัง MissionProgress Service
2. If status = RESOLVED:
 - เรียก Team Service (PATCH /status) เพื่อคืนทีมเป็น AVAILABLE
 - เรียก Request Service (POST /resolve)
3. If status = CANCELLED:
 - เรียก Request Service (POST /cancel)

Response (Success 200):

```
{
  "message": "Status updated successfully",
  "dispatchId": "uuid-string",
  "status": "DISPATCHED",
  "traceId": "trace-id-string",
  "serverTime": "2024-05-18T10:10:00Z"
}
```

Error 404: เมื่อไม่พบ dispatchId ที่ระบุ

Reliability:

- Timeout: 5s สำหรับ External Calls (Team/Request) และ 30s สำหรับ Lambda Total
- Idempotent: Yes (ใช้ Condition Expression ป้องกันการอัปเดต Record ที่ไม่มีอยู่จริง)

Async Contract

Asynchronous Function Contract

Message Contract #1: Receive Team Recommendation

ข้อมูลทั่วไป

- **Message Name:** rescue_recommendation_generated
- **Interaction Style:** Pub/Sub (Event-Driven)
- **Producer:** Team Service (AI Module)
- **Consumer:** Manage Dispatch Service
- **Channel/Queue:** RecommendationQueue (SQS) - Subscribed to Team Service SNS
- **Version:** v1

คำอธิบาย

รับข้อมูลการประเินเหตุการณ์และรายชื่อทีมกู้ภัยที่ AI แนะนำ เพื่อนามาสร้าง Record รอกการสั่งการ (Status: WAITING) และทำการดึงข้อมูลรายละเอียดทีมแบบละเอียด (Enrichment) เพื่อประกอบการตัดสินใจของเจ้าหน้าที่

Request Payload (Incoming from SQS)

```
{  
  "header": {  
    "sent_at": "ISO-8601-Timestamp",  
    "trace_id": "uuid-string"  
  },  
  "body": {  
    "request_id": "REQ-10293",  
    "request_type": "FIRE",
```

```
"priority_level": "HIGH",

"location": {

  "address_line": "123 Main St",

  "district": "Bang Rak",

  "province": "Bangkok"

},

"ranked_teams": [

  {

    "team_id": "TEAM-005",

    "rank": 1,

    "score": 0.95,

    "explanation": "ทีมอยู่ในใกล้ที่สุดและมีอุปกรณ์ดับเพลิงครบถ้วน"

  }

],

"people_count": 2,

"special_needs": "Wheelchair user"

}

}
```

Field Definition (Key Fields)

Field	Type	Required	Description
request_id	string	Y	ไอดีคำร้องขอช่วยเหลือ (ใช้เป็น dispatchId ด้วย)
ranked_teams	array	Y	รายชื่อทีมที่ AI แนะนำ พร้อมคะแนนและเหตุผล
priority_level	string	Y	ระดับความสำคัญ (เช่น HIGH, NORMAL, LOW)

Internal Action (Success)

- 1. **Data Enrichment:** ระบบจะเรียก GET /v1/teams เพื่อดึงข้อมูลเชิงลึกของทุกทีมมาเก็บไว้ใน recommendedTeams
- 2. **Database Persistence:** สร้าง Record ใน DynamoDB ด้วยสถานะ WAITING
- 3. **Sync Callback:** เรียก Request Service REST API (POST /triage) เพื่อแจ้งว่าได้รับเรื่องและประเมินเบื้องต้นเสร็จแล้ว

Message Contract #2: Dispatch Mission Publish

ข้อมูลทั่วไป

- **Message Name:** DispatchOrderCreated
- **Interaction Style:** Pub/Sub (Event-Driven)
- **Producer:** Manage Dispatch Service
- **Consumer:** Mission Progress Service
- **Channel/Topic:** request-dispatch-v1 (Amazon SNS)

- **Topic ARN:** arn:aws:sns:us-east-1:460581038623:request-dispatch-v1

- **Version:** v1

คำอธิบาย

เมื่อเจ้าหน้าที่ทำการเลือกทีมและยืนยันการส่งทีมกู้ภัย (Dispatch) ระบบจะทำการส่ง Event นี้ออกไปเพื่อให้ระบบที่เกี่ยวข้องเริ่มกระบวนการติดตามภารกิจ (Mission Progress) และจัดการทรัพยากรในลำดับถัดไป

Message Attributes (Metadata)

Name	Type	Value
messageType	String	DispatchOrderCreated

Message Body (JSON Payload)

```
{
  "header": {
    "messageType": "DispatchOrderCreated",
    "traceId": "uuid-string"
  },
  "body": {
    "dispatchId": "DSP-12345",
    "status": "DISPATCHED",
    "requestId": "REQ-10293",
    "teamId": "TEAM-005",
    "requestType": "FIRE",
```

```
"priorityLevel": "HIGH",

"evaluateReason": "ตรวจพบควันหนาแน่นผ่านระบบ IoT",

"location": "123 Main St, Bang Rak, Bangkok",

"description": "รายงานเหตุเพลิงไหม้โกดัง",

"peopleCount": 2,

"special_needs": "Wheelchair required",

"lastEvaluatedAt": "2024-05-18T10:00:00Z",

"dispatchedAt": "2024-05-18T10:05:00Z",

"timestamp": "2024-05-18T10:05:00Z"

}

}
```

Field Definition (Body)

Field	Type	Required	Description
dispatchId	string	Y	รหัสการสั่งการ (Primary Key ของระบบเรา)
status	string	Y	สถานะปัจจุบัน (จะส่งออกไปเมื่อเป็น DISPATCHED เสมอ)
requestId	string	Y	รหัสอ้างอิงคำร้องขอความช่วยเหลือจากระบบต้นทาง
teamId	string	Y	รหัสทีมกู้ภัยที่ได้รับมอบหมายภารกิจนี้

location	string	Y	สถานที่เกิดเหตุ (ที่อยู่ที่สามารถอ่านได้)
dispatchedAt	iso-8601	Y	วันที่เจ้าหน้าที่จัดส่งทีมออกไปจริง

Access Control (Permissions)

เพื่อให้ระบบภายนอกสามารถรับข้อความนี้ได้ ระบบได้รับการตั้งค่าสิทธิ์ดังนี้:

1. Allowed Consumers:

- AWS Account: Team Service
- AWS Account: Mission Progress Service

2. **Required Actions:** ต้องสร้าง SQS Queue และทำ Subscription มาที่ Topic ARN ข้างต้น โดยใช้โปรโตคอล sqs

Service Data

1) Dispatch Records (Owned by this service)

เป็นข้อมูลหลักที่เก็บใน DynamoDB (ManageDispatchTable) ซึ่งรวมข้อมูลจากการประเมิน AI และการสั่งการเข้าด้วยกัน

Field Name	Type	Required	Description	Example
dispatchId	string	Y (PK)	รหัสการสั่งการ (ปัจจุบันใช้รหัสเดียวกับ requestId)	REQ-10293
requestId	string	Y	รหัสอ้างอิงคำร้องขอความช่วยเหลือจากระบบต้นทาง	REQ-10293
teamId	string	Y	รหัสทีมที่ได้รับมอบหมาย (เริ่มต้นเป็น UNASSIGNED)	TEAM-005
status	enum	Y	สถานะของแผนงาน: WAITING, DISPATCHED, RESOLVED, CANCELLED	WAITING
priorityLevel	string	Y	ระดับความสำคัญของเหตุการณ์ (เช่น HIGH, NORMAL, LOW)	HIGH
location	string	Y	สถานที่เกิดเหตุ (ที่อยู่แบบข้อความที่ผ่านการ Parse แล้ว)	"123 Main St, Bang Rak..."

recommendedTeams	list	Y	รายชื่อทีมที่ AI แนะนำ พร้อมรายละเอียดความ สามารถและคะแนน (Enriched)	[{team_id: "...", rank: 1, ...}]
teamDetails	map	N	ข้อมูลรายละเอียดของทีมที่ ถูกเลือก (เก็บแบบ Snapshot ณ เวลาสั่งการ)	{capabilities: ["Firefighting"], ...}
statusNote	string	N	บันทึกเพิ่มเติมจากการ เปลี่ยนสถานะ (ใช้แทน completionNote)	"ส่งทีมกู้ภัยด่วน"
createdAt	datetime	Y	วันที่ได้รับข้อมูลการ ประเมินจาก AI ครั้งแรก	2024-05-18T10:0 0:00Z
updatedAt	datetime	Y	วันที่มีการแก้ไขข้อมูล ล่าสุด (ใช้แทน lastUpdatedAt)	2024-05-18T10:0 5:00Z

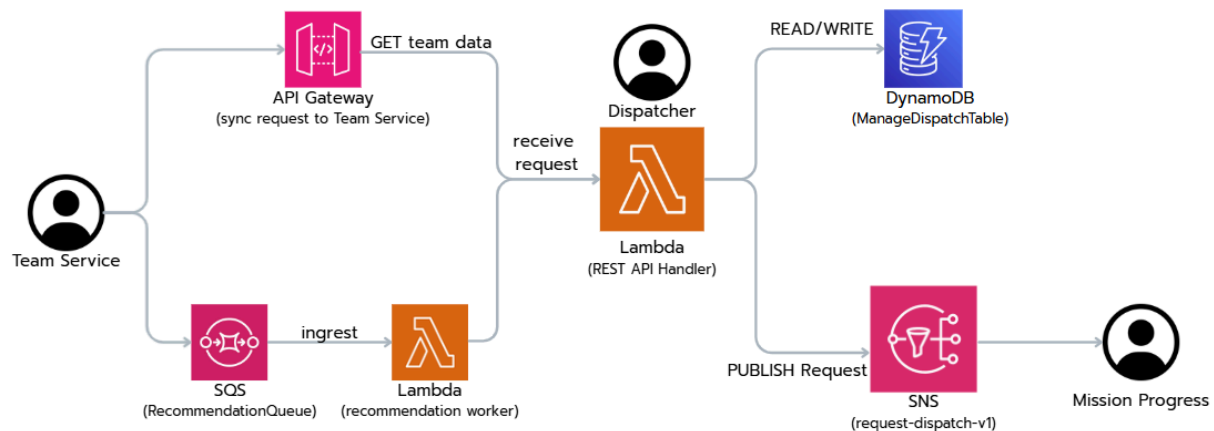
2) Mission Acceptance Status

ในระบบจริง ข้อมูลนี้ถูกจัดการผ่านการเปลี่ยนสถานะ (status) ใน Record เดียวกัน เพื่อใช้ในการควบคุมวงจรชีวิต (Lifecycle) ของภารกิจกู้ภัย

Field Name	Type	Required	Description	Example
status	enum	Y	สถานะภารกิจ: • WAITING: รอเจ้าหน้าที่สั่งการ • DISPATCHED: ยืนยันส่งทีม (เทียบเท่า ACCEPT) • RESOLVED: ภารกิจเสร็จสิ้น • CANCELLED: ยกเลิกภารกิจ	DISPATCHED
statusNote	string	N	เหตุผลประกอบการเปลี่ยนสถานะ หรือรายละเอียดรายงานหลังจบงาน	"ทีมเดินทางถึงที่ หมายแล้ว"
updatedAt	datetime	Y	วันที่สถานะระบบถูกเปลี่ยนล่าสุด	

Service Architecture

Service Architecture



Components

1. User/Client: เจ้าหน้าที่ศูนย์สั่งการ (Dispatcher) ใช้งานผ่าน Web Dashboard เพื่อตรวจสอบรายการขอความช่วยเหลือ เลือกทีมกู้ภัยที่เหมาะสมและสั่งการ (Dispatch)

2. API Gateway: จัดรับคำขอแบบ Synchronous สำหรับ Dashboard ได้แก่:

- GET /v1/dispatches: ดึงรายการ Dispatch ทั้งหมดหรือกรองตามสถานะ
- PATCH /v1/dispatches/{id}/status: อัปเดตสถานะการสั่งการ (เช่น จาก PENDING เป็น DISPATCHED หรือ RESOLVED)

3. Lambda REST API Handler (rest_handler.py): ประมวลผล Logic หลักในการจัดการคำสั่งการ โดยทำหน้าที่:

- ประสานงาน (Orchestration) เรียกใช้งาน Team Service (Sync) เพื่อตรวจสอบข้อมูลทีมหรืออัปเดตสถานะทีมเป็น BUSY/AVAILABLE
- เรียกใช้งาน Request Service (Sync) เพื่ออัปเดตสถานะของ Emergency Request (เช่น ASSIGN, RESOLVE, CANCEL)
- บันทึกสถานะและข้อมูลทีมลงใน DynamoDB
- ประกาศเหตุการณ์ (Publish Event) ผ่าน SNS เมื่อมีการสั่งการสำเร็จ

4. DynamoDB (ManageDispatchTable): ฐานข้อมูล NoSQL หลักสำหรับจัดเก็บ Dispatch Records, ข้อมูลทีมกู้ภัยที่ได้รับมอบหมายและสถานะปัจจุบันของแต่ละภารกิจ

5. Amazon SQS (RecommendationQueue): คิวรับข้อมูลคำแนะนำทีมกู้ภัย (Recommended Teams) แบบ Asynchronous ที่ส่งมาจาก Team Service (ผ่าน SNS Topic: recommendation-generated-topic)

6. Lambda Recommendation Worker (recommendation_worker.py): ทำหน้าที่ Poll ข้อความจาก RecommendationQueue เพื่อสร้าง Record การสั่งการเริ่มต้นใน DynamoDB พร้อมนำข้อมูลทีมที่ถูกยืมมาเตรียมไว้ให้ Dispatcher ดัดสินใจ (Request Enrichment)

7. Amazon SNS (request-dispatch-v1): ช่องทางประกาศเหตุการณ์ DispatchOrderCreated เมื่อ Dispatcher ยืนยันการส่งทีม เพื่อให้ MissionProgress Service นำไปสร้าง Record การติดตามภารกิจต่อ

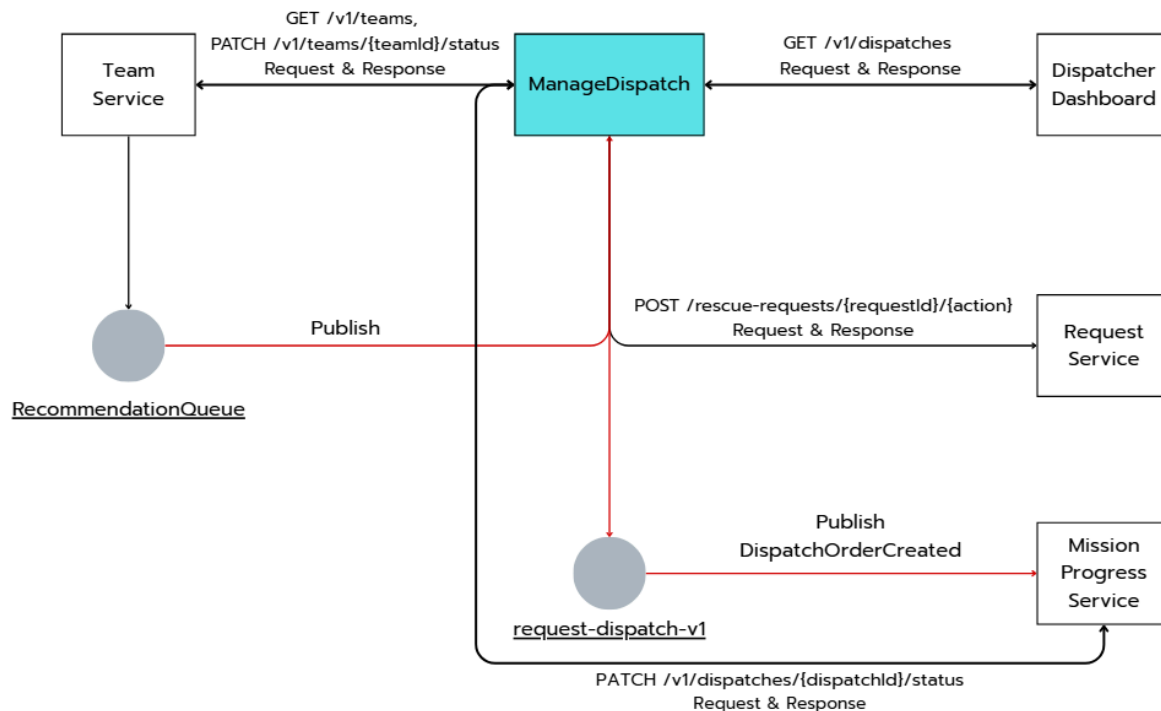
Explanation

สถาปัตยกรรมของ Manage Dispatch Service ในปัจจุบันเน้นการเป็น "Orchestrator" ที่ผสมผสานการทำงานหลายรูปแบบ:

- **Asynchronous Request Enrichment (Inbound):** เมื่อมีเหตุฉุกเฉินและระบบ Team Service แนะนำทีมที่เหมาะสม ข้อมูลจะไหลเข้าสู่ระบบผ่าน SQS (RecommendationQueue) โดยอัตโนมัติ ระบบจะดึงรายละเอียดทีมเตรียมไว้ให้ Dispatcher ทันทัน ที่ช่วยให้การตัดสินใจทำงานทำได้รวดเร็วและมีข้อมูลครบถ้วน
- **Synchronous Dispatch Execution (Control Plane):** เมื่อ Dispatcher เลือกทีมและกดสั่งการ ระบบจะทำงานแบบ Synchronous ผ่าน REST API เพื่อให้แน่ใจว่าการจองทีม (Team Service) และการอัปเดตสถานะคำขอ (Request Service) สำเร็จก่อนที่จะตอบกลับไปยังผู้ใช้เพื่อความแม่นยำของสถานะในระบบ
- **Event-Driven Integration (Outbound):** เมื่อการสั่งการเสร็จสมบูรณ์ ระบบจะประกาศเหตุการณ์ผ่าน SNS เพื่อกระตุ้นการทำงานของระบบถัดไป (MissionProgress) ตามหลักการ Loose Coupling ของ Microservices
- **Resilient Status Sync (Maintenance):** การอัปเดตสถานะกลับจากสแนม (Mission Progress) จะทำผ่านการ PATCH status กลับมายัง ManageDispatch และเปลี่ยนสถานะทีมที่ถูกยืมให้กลับมาพร้อมใช้งาน (AVAILABLE) โดยอัตโนมัติเมื่อภารกิจสิ้นสุดลง

Service Interaction

Service Interaction



Upstream Services (บริการต้นทางที่เรียกใช้งาน Manage Dispatch Service)

Dispatcher Dashboard:

- เรียกใช้งาน `GET /v1/dispatches` แบบ Synchronous เพื่อดึงข้อมูลรายการคำสั่งการทั้งหมด รวมถึงข้อมูลทีมที่ได้รับการประสานข้อมูล (Enriched Data) แล้ว เพื่อนำไปแสดงผลบนหน้าจอควบคุมให้เจ้าหน้าที่ตัดสินใจสั่งการได้ทันที

Team Service (Recommendation Module):

- ส่งคำแนะนำทีมกู้ภัย (Recommended Teams) มาในรูปแบบ Event ผ่าน SNS Topic เข้าสู่ RecommendationQueue (SQS) แบบ Asynchronous ระบบจะนำข้อมูลนี้มาสร้างเรคคอร์ดรอการสั่งการ (WAITING) พร้อมข้อมูลทีมที่ครบถ้วน

MissionProgress Service:

- เรียกใช้งาน `PATCH /v1/dispatches/{dispatchId}/status` แบบ Synchronous เพื่อแจ้งอัปเดตสถานะความคืบหน้าของภารกิจจากหน่วยงานหรือแจ้งปิดภารกิจ (RESOLVED) กลับมายังส่วนกลาง

Downstream Services (บริการปลายทางที่ Manage Dispatch Service เรียกหรือส่งข้อมูลไป)

Team Service:

- ถูกเรียกใช้งานแบบ Synchronous ผ่าน GET /v1/teams เพื่อดึงข้อมูลรายละเอียดเชิงลึกของทีม ภายใต้มาทำ Data Enrichment
- ถูกเรียกใช้งานแบบ Synchronous ผ่าน PATCH /v1/teams/{teamId}/status เพื่อเปลี่ยนสถานะทีมเป็น BUSY เมื่อมีการสั่งการและคืนสถานะเป็น AVAILABLE เมื่อภารกิจเสร็จสิ้น

Request Service:

- ถูกเรียกใช้งานแบบ Synchronous ผ่าน POST /rescue-requests/{requestId}/{action} เพื่ออัปเดตสถานะของคำร้องขอความช่วยเหลือตามวงจรชีวิตของภารกิจ (เช่น triage, assign, resolve, cancel)

MissionProgress Service (via SNS Topic):

- รับ Event DispatchOrderCreated ที่ส่งออกจาก Manage Dispatch Service แบบ Asynchronous ผ่าน SNS Topic request-dispatch-v1 เพื่อเป็นจุดเริ่มต้นในการเริ่มติดตามภารกิจในส่วนของ Mission Progress ต่อไป

Dependency Mapping

Dependency Mapping – Manage Dispatch Service

1. Team Service

- **Type:** External Service
- **Interaction Style:** Synchronous (REST)
- **Purpose:**
 - (Inbound Enrichment) ดึงรายละเอียดทีมกู้ภัยทั้งหมดเพื่อนำมาประกอบกับข้อมูลการแนะนำทีม (AI Recommendation)
 - (Status Orchestration) อัปเดตสถานะของทีมกู้ภัยเป็น BUSY เมื่อมีการสั่งการ และคืนสถานะเป็น AVAILABLE เมื่อภารกิจเสร็จสิ้น (Resolved)
- **Criticality:** Critical
- **Failure Handling:** หากบริการไม่ตอบสนอง ระบบจะบันทึก Log ข้อผิดพลาดอย่างละเอียดและอาจส่งผลให้สถานะของทีมในระบบต้นทางไม่ตรงกับความเป็นจริง (Data Inconsistency) ซึ่งต้องมีระบบตรวจสอบภายหลัง

2. Request Service

- **Type:** External Service
- **Interaction Style:** Synchronous (REST)
- **Purpose:** อัปเดตสถานะวงจรชีวิตของใบคำขอความช่วยเหลือ (Emergency Request) ให้สอดคล้องกับสถานะการสั่งการ เช่น triage (เมื่อได้รับคำแนะนำทีม), assign (เมื่อสั่งการสำเร็จ), และ resolve (เมื่อจบภารกิจ)
- **Criticality:** Critical
- **Failure Handling:** หากการเรียก API ล้มเหลว ระบบจะทำการ Retry ตามกลไกของโค้ด และบันทึกสถานะความล้มเหลวไว้ใน Log เพื่อป้องกันไม่ให้ใบคำขอ "ค้าง" อยู่ในสถานะเดิม

3. Amazon SQS (RecommendationQueue)

- **Type:** Message Queue
- **Interaction Style:** Asynchronous (Inbound Event)
- **Purpose:** รับข้อมูลชุดทีมกู้ภัยที่ผ่านการคัดเลือก (Recommended Teams) มาจาก Team Service เพื่อเริ่มต้นกระบวนการสร้างรายการเตรียมสั่งการ
- **Criticality:** Critical (สำหรับขาเข้าของข้อมูลใหม่)
- **Failure Handling:** ใช้กลไกการ Retry ของ AWS Lambda หากประมวลผลไม่สำเร็จข้อความจะถูกส่งกลับเข้าคิว (Visibility Timeout) และหากล้มเหลวซ้ำจะถูกส่งไปยัง Dead Letter Queue (DLQ) เพื่อตรวจสอบ

4. Amazon SNS (request-dispatch-v1)

- **Type:** Message Topic
- **Interaction Style:** Asynchronous (Outbound Event)
- **Purpose:** ประกาศเหตุการณ์ DispatchOrderCreated เพื่อแจ้งให้ MissionProgress Service ทราบและเริ่มกระบวนการติดตามภารกิจในภาคสนาม
- **Criticality:** High (เป็นจุดเชื่อมต่อสำคัญไปยังระบบถัดไป)
- **Failure Handling:** หากการ Publish ล้มเหลว ระบบจะพยายาม Retry ตามมาตรฐานของ AWS SNS เพื่อให้แน่ใจว่า Event จะถูกส่งถึงผู้รับ (Subscribers)

5. Amazon DynamoDB (ManageDispatchTable)

- **Type:** NoSQL Database
- **Interaction Style:** Internal Data Access
- **Purpose:** แหล่งเก็บข้อมูลความจริงชุดเดียว (Single Source of Truth) ของระบบ ได้แก่ รายละเอียดการสั่งการ, ข้อมูลทีมที่ได้รับมอบหมาย(Enriched Data), และ Timeline การอัปเดตสถานะ
- **Criticality:** Critical
- **Failure Handling:** ใช้เงื่อนไข ConditionExpression (เช่น attribute_exists) ในการอัปเดตข้อมูลเพื่อป้องกันการเขียนข้อมูลทับซ้อน (Race Condition) และรักษาความถูกต้องของข้อมูล